Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #8

Actuators – Actuation and Power Conversion. Part 2: Analog Actuator Control

---

Author:
Alexandru Cebotari
Group FAF-233

Verified by:
Alexei Martiniuc

Chisinau, 2026

Technical task

Purpose

This report documents Part 2 of the actuator laboratory. The developed application controls an analog actuator through percentage commands received from STDIO and from a potentiometer, applies analog signal conditioning, displays the current state on LCD, and reports the current values through STDIO.

Objectives

- implement analog actuator control with numeric percentage commands;
- apply saturation, median filtering, weighted averaging, and ramp limiting;
- support hardware control through a potentiometer;
- display processed values and alerts on LCD and STDIO;
- preserve a modular structure and Arduino Uno compatibility.

Individual task

The selected individual task is the analog actuator-control part of the laboratory condition. The implementation concentrates on the Part 2 scope and uses a PWM analog-output proxy suitable for simulation, validation, and reporting.

1 1. Domain analysis

1.1 1.1 Context and motivation

Analog actuator control extends the binary-control model by replacing an ON/OFF signal with a variable command. Typical embedded examples include motor speed control, dimming, or flow regulation. In such systems the raw user command must be processed before it is applied, because abrupt transitions or noise directly influence the physical output.

For this reason, the application does not simply mirror the raw input to the output. Instead, it validates, limits, filters, smooths, and ramps the command. This approach follows the practical guide and the documentation rules for the report.

1.2 1.2 Components and technologies

- Hardware: Arduino Uno, potentiometer, LCD1602 with I2C, pushbutton, PWM output proxy LED.
- Development tools: PlatformIO, Arduino framework, Wokwi, LaTeX, PlantUML.

- Software libraries: LiquidCrystal_I2C and the Arduino standard library.

The final implementation uses a PWM-driven LED in simulation as an analog actuator proxy. In a real-hardware interpretation, the same software path can control a DC motor or another analog actuator through a proper driver stage.

1.3 1.3 Existing-solution analysis

Direct PWM examples are common in introductory Arduino projects. However, they usually omit noise filtering, smoothing, and structured reporting. The chosen implementation improves on that by using an explicit processing pipeline and a layered architecture. This makes the system easier to validate, maintain, and document.

1.4 1.4 Problem definition

The application must receive analog percentage commands through STDIO or a potentiometer, process the raw values through saturation, median filtering, weighted averaging, and ramp limiting, apply the conditioned signal to the analog output at periodic recurrence, and show the resulting system state on LCD and serial output.

Section conclusion

The selected implementation strategy matches the analog-actuator scope of Part 2 and creates a suitable basis for design, implementation, validation, and reporting.

2 2. Conceptualization and design

2.1 2.1 Motivation for the design stage

The design stage transforms the laboratory statement into requirements, architectural views, behavioral flows, and validation criteria. This is necessary because analog control requires an explicit processing chain and a well-defined reporting model.

2.2 2.2 Technical requirements

ID	Requirement	Type	Acceptance target
R1	The application shall receive percentage commands through STDIO.	F	numeric commands are accepted correctly
R2	The application shall drive one analog actuator path periodically.	F	output follows processed percentage
R3	The application shall saturate raw analog commands to valid limits.	F	output remains inside allowed range
R4	The application shall apply median filtering for impulsive-noise rejection.	F	short spikes are suppressed
R5	The application shall apply weighted averaging for fluctuation reduction.	F	output is smoother than raw input
R6	The application shall apply ramp limiting for soft start and stop.	F	output changes progressively
R7	The application shall support hardware analog input through a potentiometer.	F	potentiometer can control the command path
R8	The application shall display processed values and alerts on LCD and STDIO.	F	reports are readable and periodic
R9	The system shall preserve modular software structure.	NF	responsibilities stay separated in files and folders
R10	The system shall build and run on Arduino Uno and Wokwi.	NF	successful build and simulation assets exist

Table 1: Requirements matrix for the Lab 8 analog actuator application

2.3 2.3 Architectural design

The application is structured into four logical layers:

- **Application layer**: scheduler and shared runtime state.
- **Task layer**: command, input, conditioning, actuation, and reporting tasks.
- **Service layer**: command parsing and analog signal conditioning.

- **Driver layer**: analog output, LCD, serial bridge, and hardware inputs.

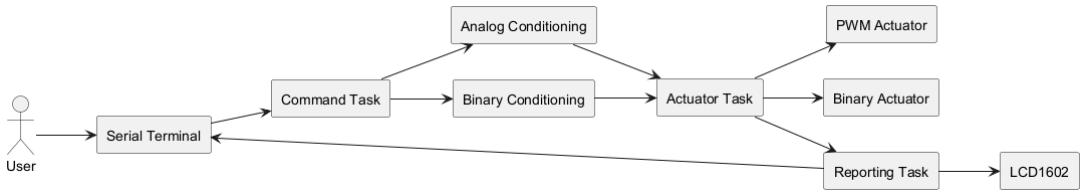

Figure 1: System structure for the analog actuator application

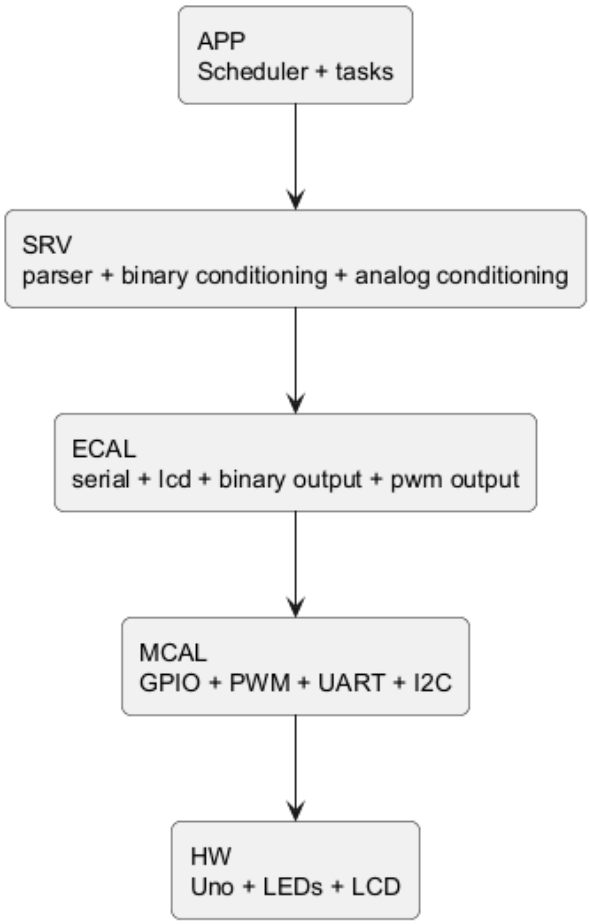

Figure 2: Hardware-software interface layers

2.4 2.4 Behavioral modeling

The analog actuator application is centered on the following processing chain:

- command acquisition from serial and hardware sources;
- saturation, median filtering, weighted averaging, and ramping;
- periodic actuation and reporting.

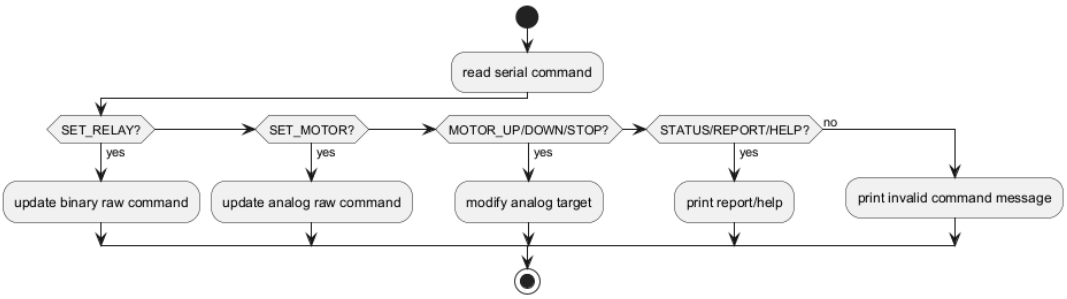

Figure 3: Command acquisition and dispatch flow

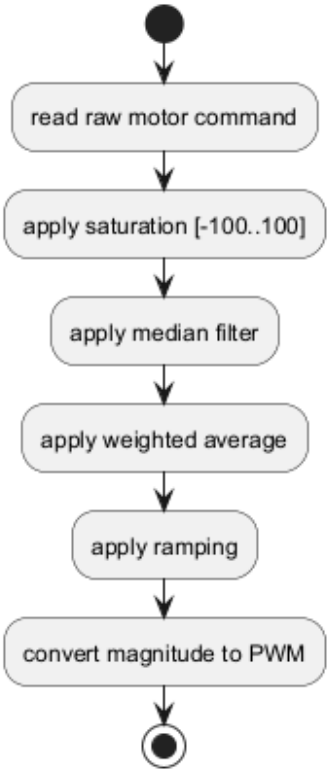

Figure 4: Analog conditioning flow

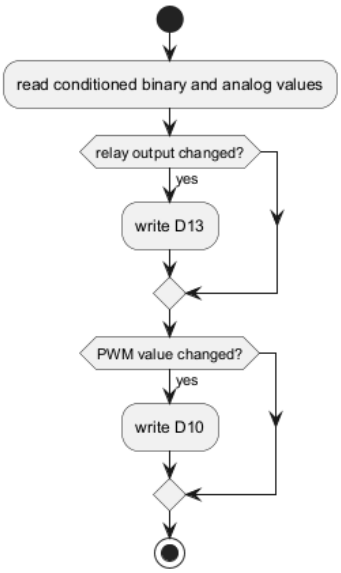

Figure 5: Actuation flow

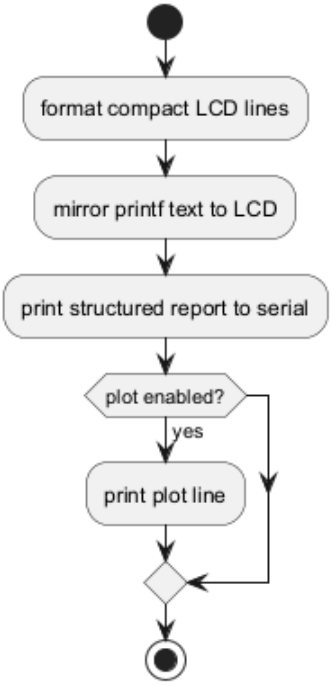

Figure 6: Reporting flow

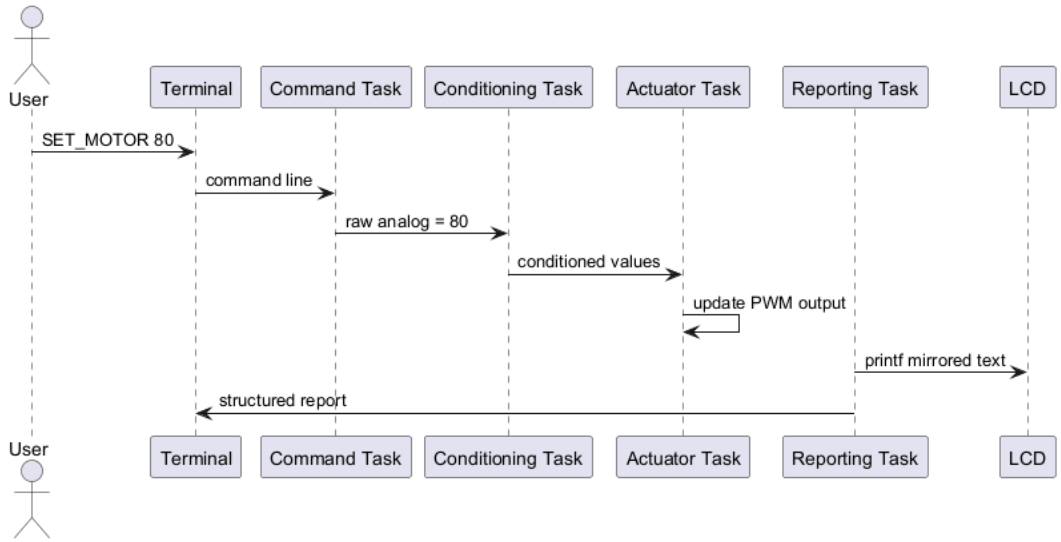

Figure 7: Runtime sequence for command, conditioning, actuation, and reporting

2.5 2.5 Validation criteria and test scenarios

ID	Scenario	Req.	Pass criterion
T1	Send valid analog percentage commands through STDIO.	R1	commands are accepted and interpreted correctly
T2	Observe output after accepted command.	R2	actuator output follows processed percentage
T3	Send values outside the allowed interval.	R3	saturation clamps command to valid range
T4	Introduce noisy or abrupt input variations.	R4	median filter suppresses spikes
T5	Apply fluctuating command values.	R5	weighted average smooths the signal
T6	Switch from low to high command values.	R6	ramp stage applies gradual transition
T7	Rotate potentiometer in hardware-enabled mode.	R7	hardware source controls the analog command
T8	Observe LCD and serial reports.	R8	processed values and alerts are visible
T9	Inspect source tree and implementation split.	R9	modular design is preserved
T10	Run `pio run` and simulation assets.	R10	project builds and simulation resources exist

Table 2: Validation scenarios for the analog actuator application

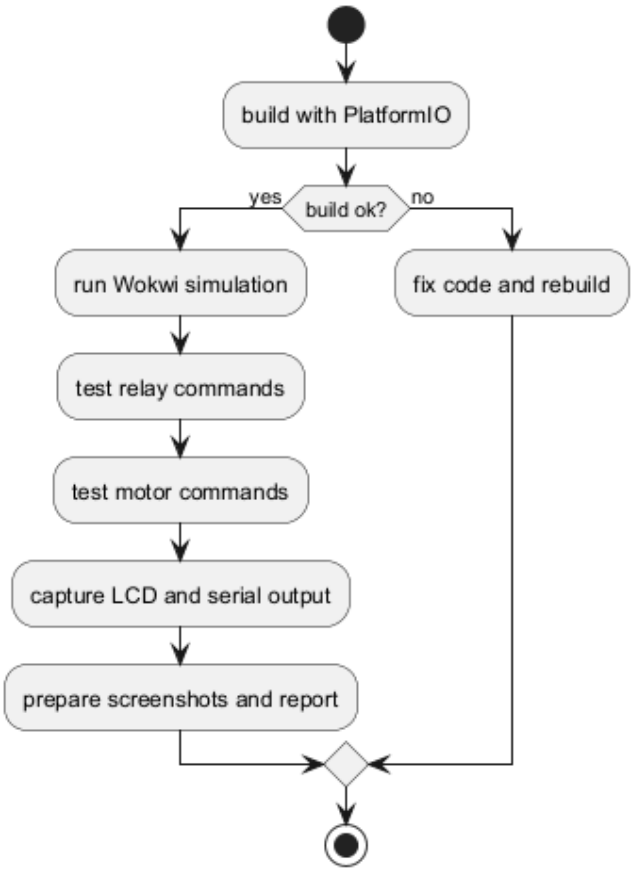

Figure 8: Validation workflow

Section conclusion

The design stage defines the technical expectations, the system architecture, the processing flow, and the validation strategy for the analog actuator solution.

3 3. Hardware and software implementation

3.1 3.1 Implemented hardware configuration

The final `Code/diagram.json` uses the following practical configuration:

- PWM analog output proxy on `D10`;
- potentiometer on `A0`;
- pushbutton on `D2`;
- LCD1602 I2C on `A4/A5`;
- serial interface through USB at 115200 baud.

The PWM output is represented by a LED in Wokwi for a clear visual demonstration. In a real circuit, the same PWM control path would be connected to a DC motor driver or another analog actuator stage.

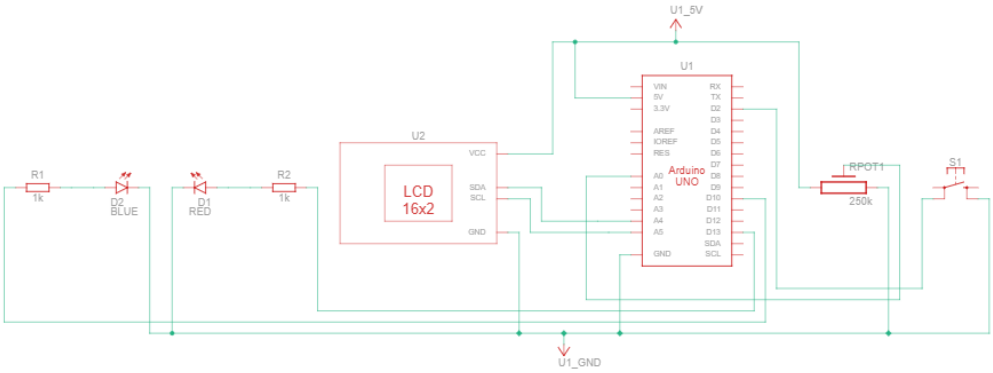

Figure 9: Electrical schematic used for the analog actuator application

3.2 3.2 Software organization

Main implementation modules are:

- `Code/src/main.cpp`: program entry point.
- `Code/src/app/app_controller.cpp`: scheduler and initialization.
- `Code/src/app/app_state.h`: shared runtime state.
- `Code/src/tasks/task_command.cpp`: serial command input.
- `Code/src/tasks/task_inputs.cpp`: hardware input sampling.
- `Code/src/tasks/task_conditioning.cpp`: analog-conditioning task.
- `Code/src/tasks/task_actuator.cpp`: output application.
- `Code/src/tasks/task_reporting.cpp`: LCD and STDIO reporting.
- `Code/src/services/analog_conditioning.cpp`: saturation, median, weighted average, and ramping.
- `Code/src/services/command_parser.cpp`: command parsing.
- `Code/src/drivers/analog_servo.cpp`: analog output abstraction.
- `Code/src/drivers/lcd_display.cpp`: buffered LCD updates.
- `Code/src/drivers/serial_stdio.cpp`: terminal output bridge.

3.3 3.3 Critical software sections

Periodic scheduling

```
ScheduledTask g_tasks[] = {
    {taskCommandRun, 20U, 0U},
    {taskInputsRun, 20U, 0U},
    {taskConditioningRun, 20U, 0U},
    {taskActuatorRun, 20U, 0U},
    {taskReportingRun, 500U, 0U},
};
```

Analog conditioning pipeline

```
g_appState.analog.saturatedPercent = clampPercent(g_appState.analog.rawPercent);
g_appState.analog.medianPercent = medianOfFive(g_medianBuffer);
g_appState.analog.weightedPercent = weightedAverage(g_averageBuffer);
g_appState.analog.conditionedPercent = applyRamp(previous, target);
```

Buffered LCD reporting

```
snprintf_P(line1, sizeof(line1), PSTR("A:%03d%% RAW:%03d"),
          toRoundedInt(g_appState.analog.appliedPercent),
          toRoundedInt(g_appState.analog.rawPercent));
lcdDisplayShowLines(line1, line2);
```

These excerpts illustrate the scheduler, the analog-signal conditioning path, and the LCD reporting mechanism, which are the most critical implementation areas for the project.

3.4 3.4 Implementation reasoning

The implementation follows the designed modular structure. The scheduler isolates periodic work into simple recurring tasks. The analog-conditioning pipeline improves stability before the PWM output is updated. LCD and STDIO reporting run at a lower recurrence than command and actuation tasks, reducing noise while preserving observability.

Section conclusion

The implementation matches the designed architecture and includes all critical elements requested by the report structure: electrical configuration, organized source tree, code excerpts, and explanation of the software decisions.

4 4. Testing and validation

4.1 4.1 Validation methodology

The application was validated through build verification, simulation observation, and functional testing:

1. compile the project with `pio run`;
2. run the Wokwi setup from the project files;
3. send serial analog commands;
4. rotate the potentiometer in hardware-enabled mode;
5. inspect LCD output, serial reports, and the analog actuator proxy;
6. compare the observed behavior to the criteria defined in Section 2.5.

4.2 4.2 Validation results

ID	Check	Status	Observation
T1	Serial analog command parsing	Pass	valid percentage commands are accepted correctly
T2	Analog output update	Pass	output follows the conditioned percentage value
T3	Saturation behavior	Pass	values outside range are clipped before actuation
T4	Median-filter effect	Pass	impulsive spikes are reduced
T5	Weighted-average effect	Pass	short fluctuations are smoothed
T6	Ramp behavior	Pass	output transitions are gradual
T7	Potentiometer hardware input	Pass	hardware analog source controls the command path
T8	LCD and STDIO reporting	Pass	percentage and alerts are clearly displayed
T9	Modularity review	Pass	code remains organized by layers and responsibilities
T10	Build and simulation	Pass	build succeeds and Wokwi assets are available

Table 3: Validation summary for the analog actuator application

4.3 4.3 Proof of operation

The report includes the standard evidence figures expected by the documentation and the practical guide:

- `compile.png`: build output from `pio run`;
- `wokwi.png`: Wokwi runtime diagram or simulation capture;
- `terminal.png`: serial terminal output;
- `hardware.jpg`: physical setup photograph;
- `electrical_schema.png`: circuit schema.

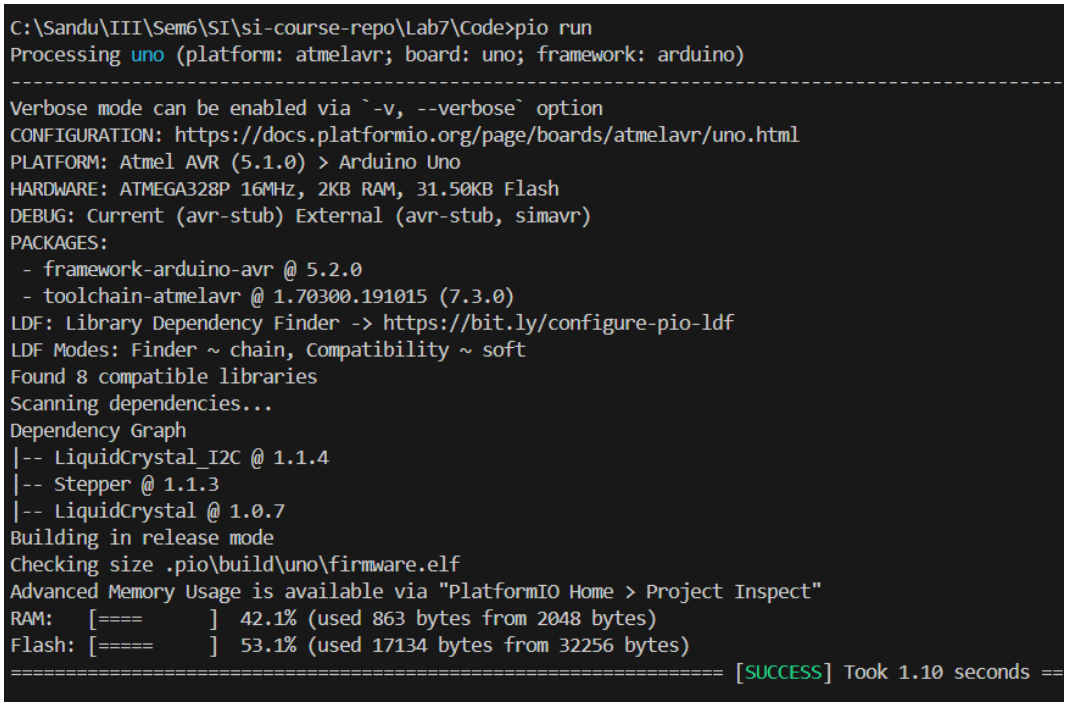

Figure 10: PlatformIO build output for the Lab 8 project

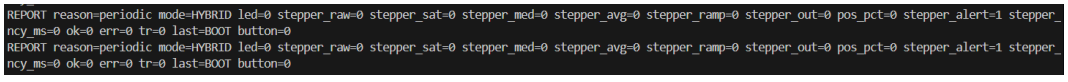

Figure 11: Terminal output for analog actuator control

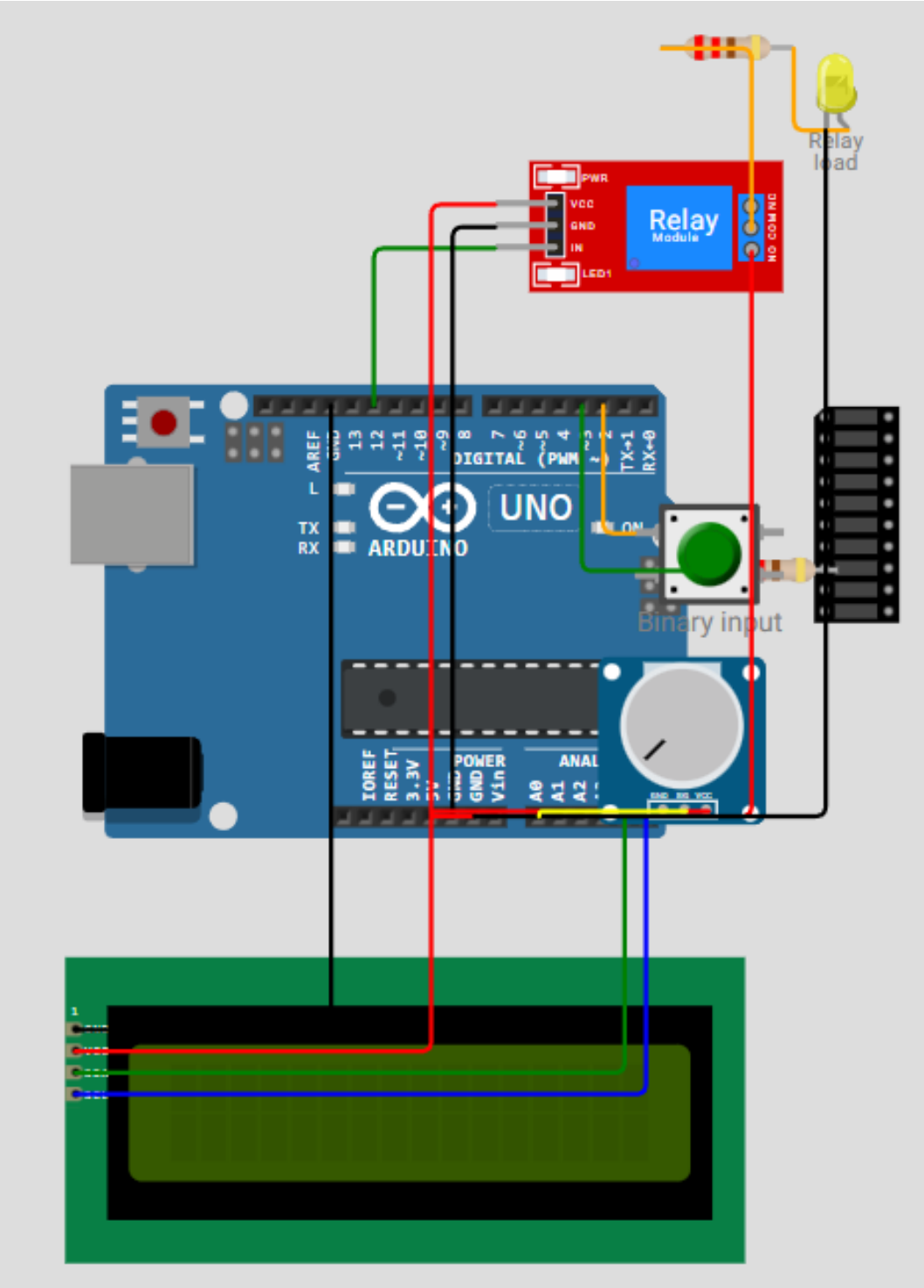

Figure 12: Wokwi runtime diagram or simulation capture

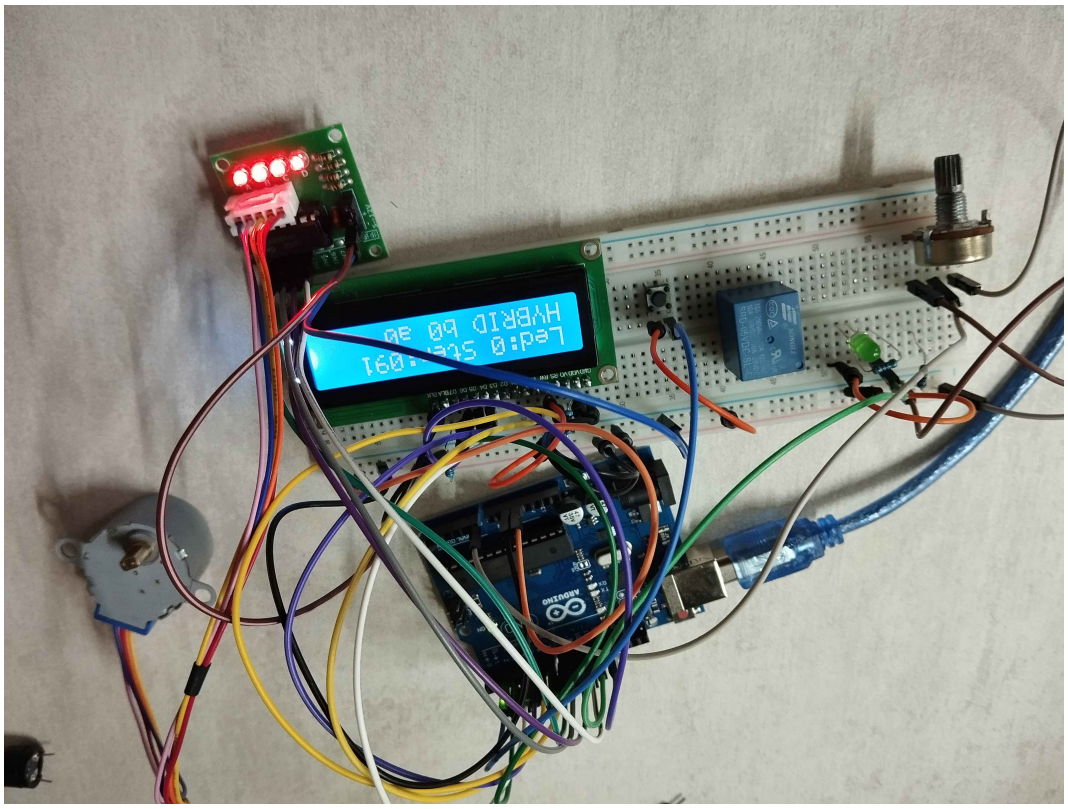

Figure 13: Hardware setup for the analog actuator project

If the final submission format asks for a video demonstration, the corresponding link can be added in this section.

4.4 4.4 Simulation observations

The Wokwi simulation confirms that the analog command can be entered through serial or potentiometer, that the conditioning stages modify the command progressively, and that the output proxy reflects the processed value. LCD and STDIO provide enough runtime information to validate the internal state of the application.

Section conclusion

Testing and validation confirm that the implemented application satisfies the Part 2 analog-actuator scope and that the expected reporting and proof-of-operation materials are present.

General conclusions

The final Lab 8 project is a complete implementation of the analog actuator task from the laboratory condition. It accepts percentage commands, conditions the raw input through multiple processing stages, applies the processed signal to the analog output path, and reports the state through LCD and STDIO.

The report follows the structure requested in the documentation: technical task, domain analysis, conceptualization and design, hardware and software implementation, testing and validation, evidence, conclusions, bibliography, and annexes. The resulting documentation is suitable for understanding, validating, and reusing the implementation.

AI tool usage note

During the preparation of this report and implementation, AI tooling was used for drafting and restructuring. The resulting material was reviewed, validated, and adjusted according to the laboratory requirements before final inclusion.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. Laboratory specification for "Actuators – Actuation and Power Conversion".
3. PlatformIO Documentation, `https://docs.platformio.org/`.
4. Wokwi Documentation, `https://docs.wokwi.com/`.
5. Arduino Language Reference, `https://www.arduino.cc/reference/en/`.
6. LiquidCrystal_I2C Library Documentation, `https://github.com/marcoschwartz/LiquidCrystal_I2C`.
7. PlantUML Language Reference, `https://plantuml.com/`.

Appendix

A. Project artifacts

The implementation is stored in `Code/`.

B. GitHub repository

Repository URL: `https://github.com/Tirppy/si-course-repo/tree/main/Lab8/Code`

C. Build and run commands

- Build: `pio run`
- Serial monitor: `pio device monitor -b 115200`
- Wokwi simulation: use `Code/diagram.json` and `Code/wokwi.toml`